

AIM2 Preparation

Week 2 - Day 3: Data Cleaning

5-Minute Review Notes | Pandas Data Cleaning

Day 3 Goal

Identify and fix common data-quality problems before analysis: missing values, duplicates, inconsistent strings, and incorrect data types.

Core Cleaning Workflow

Inspect -> Detect problems -> Clean -> Convert -> Impute/Remove -> Validate

1. Missing Values

```
df.isna()
df.isnull()
df.isna().sum()
df.notnull()
```

- **isna() / isnull()** - returns True where a value is missing and False where it exists.
- **isna().sum()** - counts missing values in each column.
- **notnull()** - returns True where values are present.

Selecting rows based on missing data

```
df[df["Score"].isna()]
df[df["Score"].notnull()]
```

2. dropna() vs fillna()

Method	Purpose
dropna()	Remove rows/columns containing missing values.
fillna(value)	Replace missing values with a chosen value.

```
average_age = df["Age"].mean()
df["Age"] = df["Age"].fillna(average_age)
```

Important: do not automatically replace every missing numerical value with 0. Zero may be a real value and can change the meaning of the data.

3. Mean Imputation and Integer Data

```
age_mean = df["Age"].mean()
age_mean = round(age_mean)
df["Age"] = df["Age"].fillna(age_mean)
```

If a field such as Age should logically be a whole number, calculate the mean first and then decide whether rounding is appropriate for the problem. Do not blindly force every mean to an integer.

4. Missing / Unknown String Values

For categorical/string columns such as City, a numerical mean is not meaningful.

```
df["City"] = df["City"].fillna("Unknown")
```

Depending on the dataset, possible strategies include:

- Use a clear placeholder such as "Unknown" when the value truly is unknown.

- Use the most frequent category (mode) only when that choice makes sense.
- Remove the row if the missing category makes the record unusable.
- Keep it missing if later analysis/modeling will handle missing categories explicitly.

5. Duplicate Rows

```
df.duplicated()
df.duplicated().sum()
df[df.duplicated()]
df = df.drop_duplicates()
```

`duplicated()` identifies repeated rows. `drop_duplicates()` removes them. Always validate afterward.

6. String Cleaning

```
df["Name"] = df["Name"].str.strip().str.title()
df["City"] = df["City"].str.strip().str.title()
```

- **`str.strip()`** removes leading/trailing spaces.
- **`str.title()`** standardizes capitalization, e.g. london -> London.
- Cleaning strings before duplicate detection/grouping helps prevent logically identical values from being treated as different categories.

7. Incorrect Data Types

```
df["Age"] = df["Age"].astype(int)
```

Values such as "28" look numeric to us, but quotation marks mean they are stored as strings. `astype(int)` works when every value can safely be converted.

8. Messy Numeric Data: `pd.to_numeric()`

```
df["Age"] = pd.to_numeric(df["Age"], errors="coerce")
```

`errors="coerce"` converts valid numeric strings to numbers and turns invalid values such as "unknown" into NaN. You can then handle the resulting missing values.

Why `astype(int)` can fail: it cannot directly convert invalid text such as "unknown" into an integer.

`pd.to_numeric(..., errors="coerce")` is safer for messy input.

9. Validation After Cleaning

```
df.info()
print(df.dtypes)
print(df.isna().sum())
print(df.duplicated().sum())
```

Cleaning is not finished when the transformation runs without an error. Verify that missing values, duplicates, and data types now match your expectations.

Day 3 Mini-Project - Messy Student Dataset Cleaner

- Inspected original data types, missing values, and duplicates.
- Cleaned Name and City using `strip()` + `title()`.
- Converted Age and Math with `pd.to_numeric(errors="coerce")`.
- Filled missing Age, Python, and AI values using column means.
- Removed duplicate student rows.
- Created Average from Math, Python, and AI with `mean(axis=1)`.
- Validated final dtypes, missing-value counts, and duplicate count.

Quick Syntax Cheat Sheet

```
df.isna()
df.isna().sum()
df.notnull()
df.dropna()
df.fillna(value)
df.duplicated()
df.drop_duplicates()
df["City"].str.strip().str.title()
df["Age"].astype(int)
pd.to_numeric(df["Age"], errors="coerce")
df[["Math", "Python", "AI"]].mean(axis=1)
```

Key Understanding Questions

isna() vs isna().sum()

isna() shows True/False for every cell; isna().sum() counts missing values per column.

Why not fill all numerical NaN with 0?

Because 0 can be a legitimate value and may incorrectly change the meaning or statistics of the dataset.

Why can astype(int) fail?

Because invalid text such as "unknown" cannot be directly converted to an integer.

Key Takeaways

- Inspect before cleaning; do not change data blindly.
- Choose missing-value treatment based on what the column represents.
- Numerical and categorical missing values often require different strategies.
- Clean inconsistent text before grouping or comparing categories.
- Use `pd.to_numeric(errors='coerce')` for messy numeric strings.
- Always validate the dataset after cleaning.

Progress

Week 2 Day 3 - Data Cleaning: COMPLETE

Next completed topic: Week 2 Day 4 - EDA / Data Analysis.